Modular Library Management System

1. Introduction

Library management is an essential task in educational institutions. Traditional manual systems are time-consuming and prone to errors such as incorrect records, difficulty in tracking issued books, and data loss. This project presents a simple and efficient Library Management System developed using Java, which automates book management, issuing, and returning processes.

---

2. Objectives

The main objectives of this project are:

- To manage library books efficiently
- To implement issue and return functionality
- To reduce manual errors
- To demonstrate Object-Oriented Programming (OOP) concepts
- To provide a modular and scalable system

---

3. Problem Statement

Manual library systems lack efficiency and accuracy. Tracking issued books, maintaining records, and updating data manually can lead to inconsistencies. This project aims to solve these issues by developing a system that automates these operations using Java.

---

4. System Architecture

The system follows a layered architecture:

- UI Layer (ConsoleUI): Handles user interaction
- Service Layer (LibraryService): Contains business logic
- Model Layer (Book, Member, BorrowRecord): Represents data
- Storage Layer (books.txt): Stores persistent data

Data Flow:
User → ConsoleUI → LibraryService → Model → File Storage

---

5. Methodology

The development of the Library Management System followed a structured approach:

1. Requirement Analysis
   The basic requirements such as adding books, issuing, returning, and viewing records were identified.
2. System Design
   A layered architecture was designed separating UI, Service, and Model components for better modularity.
3. Implementation
   The system was implemented in Java using Object-Oriented Programming principles. Classes such as Book, LibraryService, and ConsoleUI were created.
4. Integration
   All components were integrated to ensure smooth interaction between user interface, business logic, and data storage.
5. Testing
   The system was tested using different scenarios such as invalid inputs, issuing already issued books, and returning books.
6. Enhancement
   File persistence was added to ensure data is stored permanently using text file handling.

---

6. OOP Concepts Used

Encapsulation

Data members in classes like Book are private and accessed using getters and setters.

Abstraction

LibraryService hides internal logic and provides simple methods like addBook(), issueBook(), and returnBook().

Polymorphism

Different behavior is achieved based on object state, such as issued and non-issued books.

Collections

ArrayList is used to dynamically store and manage book objects.

---

7. Implementation Details

Book Class

Represents a book with attributes:

- bookId
- title
- author
- issued

Includes methods for accessing and displaying book details.

LibraryService Class

Handles core operations:

- addBook()
- displayBooks()
- issueBook()
- returnBook()

Also manages file persistence using FileWriter and BufferedReader.

ConsoleUI Class

Provides a menu-driven interface:

- Add book
- View books
- Issue book
- Return book

Handles user input and displays output.

---

8. Features of the System

- Add new books
- View all books
- Issue books
- Return books
- File-based data persistence
- Menu-driven user interface
- Input validation and exception handling

---

9. Testing & Error Handling

The system was tested with various scenarios:

- Invalid input handling (non-numeric menu input)
- Issuing already issued books
- Returning non-issued books
- Handling missing data file

Exception handling ensures the system does not crash and provides user-friendly messages.

---

10. Output Screenshots

(Add screenshots here of:)

- Main menu
- Adding a book
- Viewing books
- Issuing a book

---

11. Advantages

- Simple and user-friendly
- Modular and maintainable design
- Demonstrates OOP principles
- Supports persistent data storage

12. Limitations

- Console-based interface
- No database integration
- No authentication system

---

13. Future Scope

- GUI implementation using Java Swing or JavaFX
- Integration with database systems
- User login and authentication
- Fine calculation system

---

14. Conclusion

The project successfully demonstrates a modular library management system using Java. It applies core OOP concepts and follows a clean layered architecture. The system is efficient, scalable, and can be extended for real-world applications.